

CHAPTER 7 | TRANSACTION PROCESSING AND CONCURRENCY CONTROL

Transaction

A transaction is a sequence of one or more operations (like reading, writing, or updating data) that are executed as a single logical unit.

E.g. Transferring \$100 from Account A to Account B.

Read A.

$A = A - 100.$

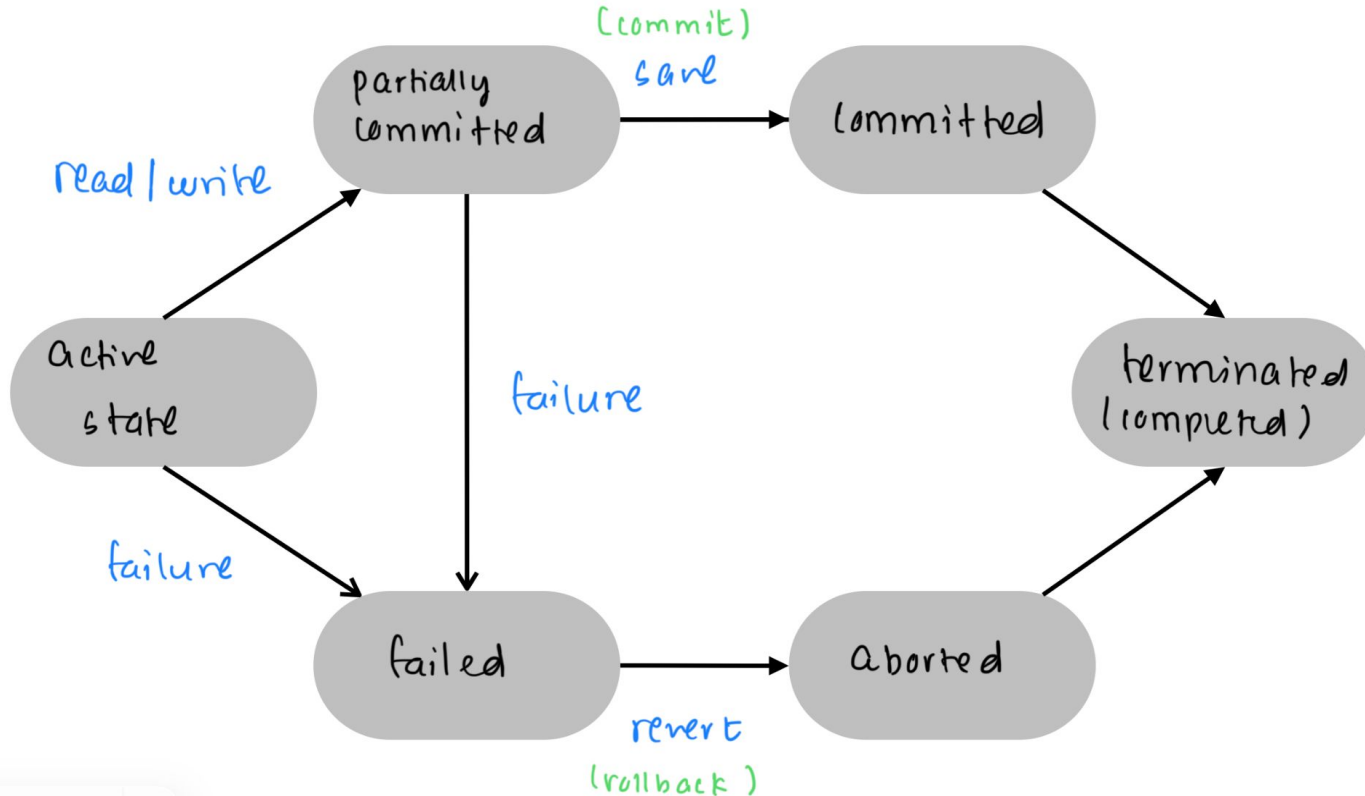
Write A.

Read B.

$B = B + 100.$

Write B.

Transaction State Diagram



Stages of a Transaction

1. *Active State*

This is the initial stage. A transaction is "Active" while its operations (Read or Write) are being executed. At this point, any changes made are stored in a temporary buffer (local memory) and are not yet visible to other users or saved permanently.

2. *Partially Committed State*

A transaction enters this state after the final operation has been executed. Even though the code is finished, the actual data is still sitting in the main memory buffer. If the power goes out now, the changes could still be lost

Stages of a Transaction

3. *Committed State*

The transaction moves here once the system confirms that all changes have been successfully written to the permanent storage (disk). The changes are now permanent and cannot be undone (except by a new, separate transaction).

4. *Failed State*

If an error occurs during the Active or Partially Committed states (like a syntax error, a system crash, or a violation of business rules like "insufficient funds"), the transaction is moved to the "Failed" state. It can no longer proceed.

Stages of a Transaction

5. *Aborted State*

Once a transaction fails, the DBMS performs a Rollback. This moves the transaction to the "Aborted" state. The goal is to undo every partial change made so that the database looks exactly as it did before the transaction started.

6. *Terminated State*

This is the "Exit" gate. A transaction reaches the Terminated state once it has finished its work (either by Committing or by being Aborted and cleaned up). The system then releases any resources or "locks" held by that transaction.

Types of transaction

In DBMS, a Schedule is the process of ordering the execution of operations (like Read/Write) from multiple transactions. Depending on how these operations are ordered, we classify them into Serial and Non-Serial transactions.

1. Serial Transactions (Serial Schedule)
2. Non-Serial Transactions (Concurrent Schedule)

Serial Transactions

In a serial schedule, transactions are executed one by one. A new transaction cannot start until the currently running one has completely finished (Committed or Aborted).

If you have Transaction T_1 and T_2 , the system will either execute $T_1 \rightarrow T_2$ or $T_2 \rightarrow T_1$.

These are always consistent because there is zero interference between users.

Concurrent Transactions

In a non-serial schedule, the operations of multiple transactions are interleaved (mixed together). The CPU switches between transactions to keep the system busy.

T₁ reads data -> CPU switches -> T₂ reads data -> CPU switches back -> T₁ writes data.

Risky. If not managed properly, it leads to the challenges like Lost Updates or Dirty Reads.

ACID properties

To ensure reliability, every transaction must follow the ACID properties. This is the most important set of keywords in transaction management.

Keyword	Meaning
Atomicity	The "all or nothing" property. A transaction is an indivisible unit.
Consistency	The database must remain in a valid state before and after the transaction (e.g., total money in a bank remains constant).
Isolation	Each transaction should execute as if it is the only one running, even if many are happening at once.
Durability	Once a transaction is "Committed," the changes are permanent and won't be lost even if the system crashes.

Concurrent Execution of Transactions

Concurrent execution occurs when the operations of multiple transactions (like Read and Write) are interleaved. Instead of finishing Transaction A entirely before starting Transaction B (Serial Execution), the CPU switches between them.

- Serial Execution:
T1 → T2 → T3 (One after another)
- Concurrent Execution:
T1(Read) → T2(Read) → T1(Write) → T2(Write) (Interleaved)

Serializability

Serializability is the criterion used to determine if a concurrent (non-serial) schedule is "correct." A schedule is serializable if its final outcome is the same as if the transactions had been executed one after another in some order.

When multiple people or programs are reading and writing data at the same time, the end result is exactly as if all those operations had happened one at a time, in some order.

It's types are:

1. Conflict Serializability
2. View Serializability

Conflict Serializability

- A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting, adjacent operations.
- Conflict Definition: Two operations conflict if they:
 - Belong to different transactions.
 - Access the same data item (X).
 - At least one is a Write (W(X)).
- Condition: A schedule is conflict serializable if its Precedence Graph (where transactions are nodes and conflicting operations are directed edges) has no cycles.

Conflict Serializability Example

Both read $X=100$, then both write. Last write wins—one update is lost.

Therefore,

A conflict occurs when two operations:

- Belong to different transactions
- Access the same data item
- At least one is a write

Time	Transaction A	Transaction B
1	$R_1(X)$	
2		$R_2(X)$
3	$W_1(X)$	
4		$W_2(X)$

Conflict Serializability Example

1. Create nodes for both transactions
2. For any conflict extend an arrow from the leading node to the following node.
E.g. Here, A $\rightarrow$ B (due to 1) &
B $\rightarrow$ A (due to 2)

Which is cyclic in nature, hence the transaction is not conflict serializable.

Time	Transaction A	Transaction B
1	R ₁ (X)	
2		R ₂ (X)
3	W ₁ (X)	
4		W ₂ (X)

Testing for Conflict Serializability

Construct a precedence graph:

- Node = transaction
- Edge $A \rightarrow B$ = A must come before B (conflict exists)

$A \rightarrow B$ (because A reads X before B writes X)

$A \rightarrow B$ (because A writes X before B writes X)

If the graph has no cycles, it's conflict serializable.

View Serializability

- It is a more relaxed (broader) type of serializability.
- View Equivalence: Two schedules S and S' are view equivalent if:
 - Initial Read: If T_i reads data X first in S , it must read X first in S' .
 - Updated Read: If T_i reads a value of X written by T_j in S , it must do the same in S' .
 - Final Write: The transaction that performs the final write on X in S must also perform the final write in S' .
- Relationship: Every conflict serializable schedule is also view serializable. However, a schedule that is not conflict serializable can still be view serializable if it contains Blind Writes (writing data without reading it first).

Rules of View Serializability

- Rule 1: Initial Read Rule
 - If a transaction reads the initial value of a data item in the schedule,
 - it must also read the initial value in the equivalent serial schedule.
- Rule 2: Read-From Rule
 - If a transaction reads a value written by another transaction,
 - it must read from the same transaction in the serial schedule.
- Rule 3: Final Write Rule
 - The transaction that performs the final write on a data item
 - must be the same in both the schedule and the serial schedule.

Justification of rules of View Serializability

- Rule 1: Initial Read Rule

T1 reads initial value of X.

In serial order $T1 \rightarrow T2$, T1 also reads initial value.

- Rule 2: Read-From Rule

T2 reads the value written by T1.

In serial order $T1 \rightarrow T2$, T2 also reads from T1.

- Rule 3: Final Write Rule

Final write is by T2.

In serial order $T1 \rightarrow T2$, final write is also by T2.

Time	Transaction A	Transaction B
1	R1(X)	
2	W1(X)	
3		R2(X)
4		W2(X)

Diagnostics vs Implementation

Serializability is diagnostic: it checks if a completed schedule was "safe" by looking for cycles in a precedence graph.

However, a database cannot wait for transactions to finish before deciding if they were okay. It needs a preventative mechanism to ensure cycles never form in the first place. Locks are the proactive tool used to enforce this order in real-time.

Locks

Locks are digital permissions (variables) that a transaction must acquire before it can access or modify a data item.

A lock is a status flag associated with a data item (like a row or table). It informs the database whether a piece of data is currently "busy" and what kind of operations are allowed on it.

It's types are:

1. Shared Lock
2. Exclusive Lock

Shared Lock

- Purpose: Used for Reading.
- Behavior: Multiple transactions can hold a Shared Lock on the same data at the same time (Shared access).
- Restriction: While a transaction holds an S-lock, no other transaction can Write to that data.

Exclusive Lock

- Purpose: Used for Writing (Insert, Update, Delete).
- Behavior: Only one transaction can hold an X-lock on a data item at a time.
- Restriction: While a transaction holds an X-lock, no other transaction can Read or Write to that data.

The Lock Compatibility Matrix

In a lock-based protocol, the database uses a compatibility matrix to determine whether a transaction request for a lock on a data item can be granted immediately or if it must wait.

Current Lock	Requested: Shared (S)	Requested: Exclusive (X)
None	Granted	Granted
Shared (S)	Granted	Wait
Exclusive (X)	Wait	Wait

Two Phase Locking (2PL)

- It is the standard protocol used by databases to ensure that schedules are Conflict Serializable in real-time.
- It acts as a set of rules that prevents transactions from interfering with each other in ways that would create cycles in a precedence graph.
- The central rule of 2PL is: **Once a transaction releases any lock, it can never acquire another new lock.**

2PL | Growing Phase

- The transaction may acquire new locks (Shared or Exclusive).
- It can upgrade a lock (e.g., change a Shared lock to an Exclusive lock).
- Crucially: It cannot release any locks during this phase.

2PL | Shrinking Phase

- The transaction may release existing locks.
- It can downgrade a lock (e.g., change Exclusive to Shared).
- Crucially: It cannot acquire any new locks.

Time	Transaction T1	Transaction T2	State / Result
1	Lock-S(A)		T1 starts Growing Phase.
2	Read(A)		
3	Lock-X(B)		Lock Point for T1 (All locks acquired).
4	Write(B)		
5	Unlock(A)		T1 starts Shrinking Phase.
6		Lock-S(A)	T2 starts Growing Phase (gets A since T1 released it).
7	Unlock(B)		T1 is finished.
8		Lock-X(B)	T2 gets B. Lock Point for T2.
9		Write(B)	
10		Unlock(A)	T2 starts Shrinking Phase.
11		Unlock(B)	T2 is finished.

Time	Transaction T1	Transaction T2	State / Result
1	Lock-S(A)		T1 gets read access to A.
2	Read(A)	Lock-S(B)	T2 gets read access to B.
3		Read(B)	Both transactions are in the Growing Phase.
4	Lock-X(B)		WAIT: T2 holds a lock on B. T1 is blocked.
5	(Waiting...)	Lock-X(A)	T2 needs A, but T1 holds it.

DEADLOCK!!

DEADLOCK

A deadlock in a DBMS occurs when two or more transactions are stuck in a circular "waiting" state, where each holds a resource the other needs.

Because neither can proceed without the other's resource, they wait indefinitely.

It can be prevented using:

1. Wait-Die
2. Wound-Wait methods

Wait-Die

- Rule: Older can wait, younger must die. If transaction T_i requests a resource held by T_j :
 - Older requests Younger:
If $TS(T_i) < TS(T_j)$, T_i is allowed to wait.
 - Younger requests Older:
If $TS(T_i) > TS(T_j)$, T_i must die (abort and restart).

Requesting Transaction	Holding Transaction	Relationship	Action
T1 (10s) requests from	T2 (20s)	Older requests Younger	T1 Waits
T1 (10s) requests from	T3 (30s)	Older requests Younger	T1 Waits
T2 (20s) requests from	T1 (10s)	Younger requests Older	T2 Dies
T3 (30s) requests from	T1 (10s)	Younger requests Older	T3 Dies

Wound-Wait

- Rule: Older wounds younger, younger must wait. If transaction T_i requests a resource held by T_j :
 - Older requests Younger:
If $TS(T_i) < TS(T_j)$, T_i wounds T_j (T_j aborts and restarts).
 - Younger requests Older:
If $TS(T_i) > TS(T_j)$, T_i is allowed to wait.

Requesting Transaction	Holding Transaction	Relationship	Action
T1 (10s) requests from	T2 (20s)	Older requests Younger	T1 Wounds T2 (T2 aborts)
T1 (10s) requests from	T3 (30s)	Older requests Younger	T1 Wounds T3 (T3 aborts)
T2 (20s) requests from	T1 (10s)	Younger requests Older	T2 Waits
T3 (30s) requests from	T2 (20s)	Younger requests Older	T3 Waits

Multiple Granularity

In database management systems, Multiple Granularity refers to breaking the database into a hierarchy of differently sized "chunks" (granules) that can be locked independently.

Instead of only locking a whole database or only locking a single row, multiple granularity allows the system to choose the most efficient size for the task at hand.

The Granularity Hierarchy

The database is organized like a tree. If you lock a node at a higher level, everything below it (its descendants) is automatically locked in the same way.

- Database (Highest level)
- Areas/Files
- Tables
- Pages/Blocks
- Records/Rows (Lowest level)

Coarse Granularity

Locking an entire Database or Table.

It requires very little "paperwork" (overhead) for the CPU to track one big lock. However, it kills concurrency because if one person is editing a single row, the entire table is blocked for everyone else.

- Pro: Low overhead (only one lock to track).
- Con: Low concurrency (one user locks the whole table, everyone else waits).

Fine Granularity

Locking individual Rows or Records.

It maximizes concurrency because many users can edit different rows in the same table simultaneously. However, it creates high overhead because the system must track thousands of individual locks.

- Pro: High concurrency (many users can access different rows of the same table).
- Con: High overhead (the system must manage thousands of individual locks).